

# Creating a Dual-Agility Method: The Value of Method Engineering

B. Henderson-Sellers, University of Technology, Australia  
M.K. Serour, University of Technology, Australia

---

## ABSTRACT

*Agile approaches to software development provide flexibility within the method, but provide little or no flexibility to support software process improvement (SPI). This second kind of flexibility is important to permit organizations to improve with time. To accomplish such improvement, a method-engineering approach is shown here to be useful. Examples of the application of method engineering to agile method construction are given based on the use of the OPEN process framework. A dual-agility method to support both intraproject and interproject flexibility can then be constructed by the selection of appropriate method fragments from this repository with the use of the accompanying guidelines. This is illustrated in an industry study of an organization transitioning to an agile software-development environment in which a minimalistic agile method was first constructed and then enhancements were made as the organization itself improved the process.*

**Keywords:** *action research; agile methods; dual-agility methods; metamodeling; method engineering; software process improvement; the OPEN Process Framework*

---

## INTRODUCTION

In the context of software (and indeed systems) development, it has long been recognized that it is not possible to design or select a single method for all conceivable situations (Cockburn, 2000; Turk, France, & Rumpe, in press), especially for all prevailing organizational cultures (Constantine & Lockwood, 1994). Despite this obvious truism, methodologists over the last two to three decades have created all-encompassing methods that purport to of-

fer “the solution” (see, e.g., the discussion in Abrahamsson, Warsta, Siponen, & Ronkainen, 2003). Chau, Maurer, and Melnik (2003) argue that this resulted from the adoption of Tayloristic (1998) methods from the manufacturing domain into the software-development domain: methods that shift the focus from people to an overinflated importance of process for its own sake. Many of these large methods have been marketed by large consulting firms and others have been published in books for direct client adoption.

As a reaction to these large, so-called “heavyweight” methods, both in structured development and, more recently, in object-oriented development, the last few years have seen the rise of a group of lightweight or agile methods (such as Crystal, Scrum, XP) as offering more flexibility, particularly in situations of unstable requirements (e.g., Bettin, 2003; Newkirk, 2002) and in a reversion to a focus on the people involved in software development. These agile methods have, to date, been largely tried on small projects, and there is significant debate as to what extent they will or will not scale up (see, for example, Constantine, 2002; Reifer, Maurer, & Erdogmus, 2003). However, the research literature is scant (see, e.g., Abrahamsson et al., 2003) and many success stories are anecdotal (Turk et al., in press). Indeed, Lindvall et al. (2002) identify “an urgent need to empirically assess the applicability of these methods.” While these newer agile methodologies offer flexibility and responsiveness, they are static in that they offer little or no support for large-scale process improvement in the sense of CMM or SPICE.

Interestingly, although software process improvement (SPI) addresses issues of moving from one software maturity level to another, its focus is on assessment, not on the facilitation of that advancement. Consequently, the process needed to effect SPI is rarely discussed in the SPI literature (e.g., Jalote, 2002). Indeed, in a questionnaire survey by Rainer and Hall (2002), no questions were asked about these process maturation topics. As we shall show, one possible underpinning technology for SPI could be method engineering (ME): There has to date been no intersection of ME ideas and SPI ideas.

In this article, we evaluate these contradictory demands for a good software-development method: the freedom for software developers to be creative and not stifled by bureaucracy on a specific project vs. the need for a standard approach to developing software such that organizational learning and SPI might flourish. In the context of software engineering, a viable development method needs to be comprehensive enough to support the entire devel-

opment cycle even if it is required to provide flexibility or agility. Agile methods are not designed nor expected to be incomplete or functionally disabled. Instead, they should add more features and functionality to existing methods in order to be able to handle the ever-changing world of requirements.

By examining the limitations of most of the existing lightweight and agile methods, we can clearly state that there is a real need for agile methods that support flexibility within the method’s application to a specific project (a.k.a. enactment), and also that support SPI, preferably underpinned by ME, that is, agility in two dimensions. We name these dual-agility methods, created by adding even more agility to the existing agile methods to be able to offer more flexibility, applicability, and functionality as organizations evolve and mature. Such a dual-agility method is engineered to be agile within a single project and also to offer agility across multiple projects so that the organization can also reap the benefits of software process improvement.

In this article, we propose a way of accomplishing this second kind of agility by the adoption of a method-engineering approach using the metamodel-based OPEN process framework or OPF (Firesmith & Henderson-Sellers, 2002). In the next section, we first review some variants on the notion of agility and then, in the subsequent section, identify the requirements for an agile methodology in an evolving organization. In the section titled “Method Engineering,” we begin to seek a solution in terms of method engineering, particularly by the use of the OPF. In “Empirical Evaluation,” we present a preliminary empirical evaluation before summarizing and concluding.

## AGILITY VARIATIONS

### Agile or Lightweight?

Many software methodologists and developers use the term agile and lightweight interchangeably. Here, we argue that the two words are not the same, and we should select the most appropriate word for our purpose to avoid any ambiguity.

*Table 1. The principles of the Agile Manifesto*

|                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Our highest priority is to satisfy the customer through the early and continuous delivery of valuable software.                         |
| 2. Business people and developers must work together daily throughout the project.                                                         |
| 3. We welcome changing requirements, even late in development.                                                                             |
| 4. We deliver working software frequently.                                                                                                 |
| 5. Working software is the primary measure of progress.                                                                                    |
| 6. We build projects around motivated individuals. We give them the environment and support they need, and trust them to get the job done. |
| 7. The best architectures, requirements, and designs emerge from self-organizing teams.                                                    |
| 8. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.         |
| 9. Agile processes promote sustainable development.                                                                                        |
| 10. Continuous attention to technical excellence and good design enhances agility.                                                         |
| 11. Simplicity is essential.                                                                                                               |
| 12. Project teams evaluate their effectiveness at regular intervals and adjust their behavior accordingly.                                 |

The term lightweight, more often than not, is used to describe a partial, incomplete, or downsized version of a full-sized (or heavyweight) product. Therefore, to the human mind, lightweight implies a product with time limitations and/or some omitted features and/or functionalities in order to make the product more affordable. For example, a lightweight software system could be limited by the number of records that can be saved, or it could be limited by the number of users that can access the system. Consequently, people, with their natural sociological behavior, assess the lightweight as always being second best. As a result, they usually build only partial trust in lightweight products compared to building full trust in the full and complete heavyweight products.

Agility, on the other hand, denotes flexibility: readiness for action or change (Abrahamsson et al., 2003). Agile teams, often cross-functional, tend to be small (e.g., Chau et al., 2003), fast moving, and highly reactive to changes, especially in requirements. Communication between team members tends to be informal rather than written, and

tacit knowledge is emphasized at the expense of explicit knowledge (e.g., Chau et al.). A number of implementations of the Agile Manifesto (see also Tables 1 and 2), such as XP, ASD, and Crystal, are reviewed objectively by Abrahamsson et al. using an evaluation framework of five perspectives (Table 3). Agile methods tend to focus on people, a small but disciplined process (e.g., Beck & Boehm, 2003), code as a deliverable, and minimal (or zero) documentation. Cockburn (2003) stresses that an agile method is not any specific set of techniques, but rather is a mind-set (see also Ambler, 2003). For example, XP is based on the following key values of

- Communication,
- Feedback,
- Simplicity, and
- Responsibility,

together with a set of 12 basic practices (Beck, 2000). This suggests that agility is of more utility than a lightweight methodology.

Table 2. The four comparative values of the “Manifesto for Agile Software Development” (<http://www.agilemanifesto.org>; characteristics on the left are preferred to those on the right)

|                              |    |                             |
|------------------------------|----|-----------------------------|
| Individuals and interactions | cf | Processes and tools         |
| Working software             | cf | Comprehensive documentation |
| Customer collaboration       | cf | Contract negotiation        |
| Responding to change         | cf | Following a predefined plan |

Table 3. Five perspectives for analyzing agile methodologies, proposed by Abrahamsson et al. (2003)

|                                                    |
|----------------------------------------------------|
| * Software development cycle                       |
| * Project management                               |
| * Abstract principles vs. concrete guidance        |
| * Universally predefined vs. situation appropriate |
| * Empirical evidence                               |

**Single Agility Within  
a Single Method**

Fowler (2002) argues that lightweight or agile<sup>1</sup> approaches are necessary because the existing and dominant methods have too much overhead and are resource hungry. Indeed, many authors cite the popularity of the CMM (and now the CMMI) and ISO12207 as leading to organizations creating and using processes that are more heavy than necessary.

It is argued that existing heavyweight processes tend to be predictive and slow, following Taylor’s (1998) approach to manufacturing processes (Chau et al., 2003). They are thus unable to react to changing circumstances, but are useful for fixed-price and/or fixed-scope contracts, particularly where requirements are well known. They can also be used when significant project-management overhead must be incurred, for instance, with large, business-critical systems built by teams often in excess of 50 members (Fowler, 2002). In contrast, lightweight or agile methods are seen to be more code oriented, more people

oriented, and more able to adapt to changes than the traditional approaches. The use of an agile method on a relatively small project with rapidly changing requirements would thus seem to be ideal. With XP, for example (Beck, 2000), there are a number of prespecified practices and values. Advocates of XP state publicly that if any one of these is missing, then it is inappropriate for the development team to claim they are using XP. In other words, while the method itself provides a high degree of flexibility to react to customers’ whims, the overall project-management element and the very structure of the procedures themselves tend to be tightly controlled, as indeed they must be if such a rapid, high-pressure development can succeed. Highsmith (2000) highlights the use of an agile method in a high-risk environment always on the edge of the abyss (he uses a mountaineering metaphor to explain his particular style of development using an agile method). With such tight control and such high-risk ventures, the process would appear to be in what the physicists would call unstable equilibrium, needing careful management. Fur-

Copyright © 2005, Idea Group Inc. Copying or distributing in print or electronic forms without written permission of Idea Group Inc. is prohibited.

Table 4. Limitations of agile processes identified by Turk et al. (in press)

|                                                          |
|----------------------------------------------------------|
| Limited support for distributed development environments |
| Limited support for subcontracting                       |
| Limited support for building reusable artifacts          |
| Limited support for development involving large teams    |
| Limited support for developing safety-critical software  |
| Limited support for developing large, complex software   |

thermore, even for a highly skilled team that successfully maintains that balance, a second challenge arises if and when the development team needs to bring their knowledge to other situations, or to inculcate an agile mind-set on new team members. In other words, the careful and high degree of tuning to one particular context is unlikely to transfer to another context without serious and extensive retuning.

### Weaknesses with Contemporary Agile Methods

A number of weaknesses or limitations are identified by Turk et al. (in press; see Table 4), while Skowronski (2004) argues that people with good problem-solving skills will be sidelined in favor of those with poorer analytical skills. Additionally, existing agile methods can offer only one dimension of agility that manifests itself in the degree to which the flexibility can handle the requirement instability and respond and react to customers rapidly. While each agile method is highly flexible in that it permits potentially extensive changes to requirements, it still imposes some significant amount of rigor on the process component of the method<sup>2</sup>. Thus, within the selected method exercised on a single project, the developers find agility and are encouraged by the principles of the Agile Manifesto (<http://agilemanifesto.org/principles.html>) to reflect upon their team effectiveness and to seek to improve as necessary (e.g., Kerievsky, 2001; Kerth,

2001). However, when moving from project to project in different domains with varying project sizes and scopes in different organizations, it is unlikely that the same agile method will be appropriate (Henninger, Ivaturi, Nuli, & Thirunavukkaras, 2002). Most, if not all, existing agile methods are designed to only work with a set of predefined values and prespecified practices. If developers are not able to secure all these preconditions, obviously, they cannot adopt and implement the specific agile method. This means that knowledge transfer may well be limited and we (the software-development community) end up with a whole suite of disconnected instances of agile methods.

Agile methods today parallel object-oriented methods in the early 1990s: There are several to choose from but there is little migration possible between them (Abrahamsson et al., 2003). Each has a clearly defined niche. Within that niche, it can be highly effective; but for an organization wishing to grow (in maturity) over the next few years, there is no clear improvement path (Henninger et al., 2002).

One of the ideas promulgated by the Software Engineering Institute (SEI) and others (e.g., ISO 15504) is that of continuously improving process and method usage. This is known generically as software process improvement, and it is argued that the maturity of the software-development organization (or more likely, each team) can be very different from the matu-

rity of other software-development organizations and can therefore change in time. This leads to the concept, embodied in CMM, CMMI, SPICE, and OOSPICE, of a set of levels of attainment or capability. These levels (usually 1 to 5, or 0 to 5) reflect the capability of the organization either overall (the CMM approach) or for each individual process<sup>3</sup> (the SPICE approach). The measured capability reflects the success rate for the specific process as enacted on a specific project and thus can be used to identify weaknesses and suggest areas for improvement (both technical and managerial).

The inability to scale is another major weakness of today's agile methods as most of the agile methods are being designed to work with relatively small projects and with a limited number of developers. If the current project expands, the implemented agile method will not be able to scale up to handle the new situation (Reifer et al., 2003).

## DUAL AGILITY

As a result of the investigation of the weaknesses of contemporary agile methods, we need to engineer a method that can react rapidly to changes internal to the project and also to longer term environmental changes. A dual-agility method should be able to serve the two most needed flexibility demands: agility to changes and agility to self-tuning and/or SPI. The first agility or flexibility dimension makes the method more able to adapt to changes not only for requirements but also for design, technology, and people. This flexibility dimension makes the method more people focused instead of function focused and therefore requires a continuous collaboration with customers.

The second agility dimension makes the method more flexible so that it can be changed or reengineered if and when a need arises in response to organizational evolution and maturation. When a new project requires extra activities or tasks (method fragments), the development team should be able to fine-tune or reengineer their method to fully support the changed demands of the project. In other words,

a dual-agility method can be created at the organizational level and then fine-tuned or instantiated for each specific project (called adaptation by Aydin, Harmsen, van Slooten, & Stegwee, 2004).

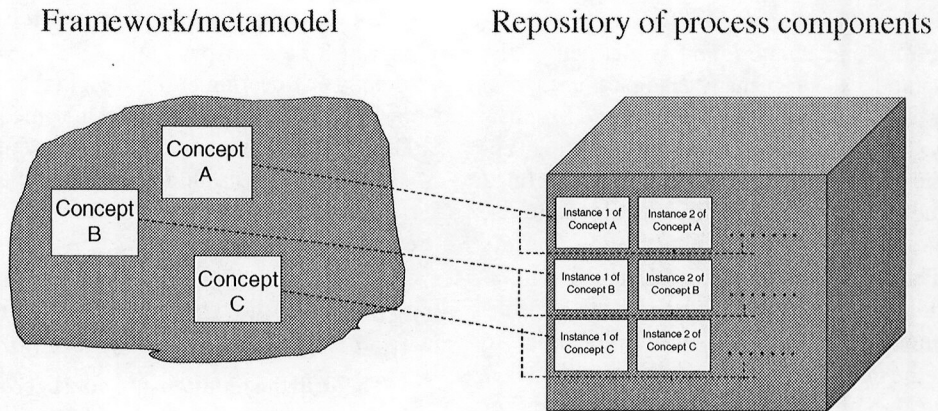
Engineering a dual-agility method using a rich repository of method fragments such as are found in the OPF or in more recent versions of the RUP (MacIsaac, 2003) can assure a high degree of flexibility to quickly plug and unplug process components in order to reengineer or fine-tune the agile method when it is required.

As a result, our proposed dual-agility method will be able to scale up with project size, expanding life cycle activities and adapting to any environmental changes. A dual-agility method must be able to offer a high degree of flexibility in order to react or respond quickly to changes in requirements, technology, or personnel as well as possessing a second degree of flexibility pertinent to scalability and tailorability. A dual-agility method must be flexible enough to be reengineered to suit different projects, organizations, or even domains. A dual-agility method must be able to adapt rapidly to requirement changes even at later stage, and environmental changes such as the project size, scope, and domain.

A dual-agility method must be engineered with people in mind; it is not a set of techniques but rather a new approach or way of thinking about software development. It is very much people focused. Developers own their method and they self-tune it when necessary. Customers become an effective and vital partner throughout the whole development life cycle, as is the case for all agile methods (e.g., Oppertthausen, 2003).

To support evolutionary agility (i.e., the agility of the methodological approach itself rather than the elements within it, as discussed in "Agility Variations"), we need to include in the method some mechanism by which the method itself can change. One highly promising approach is the use of method engineering, as discussed in subsequent paragraphs.

Figure 1. The generation of instances from the metamodel/framework to be placed into the component repository (adapted from Firesmith & Henderson-Sellers, 2002)



## METHOD ENGINEERING

It is neither viable nor practical for an organization to adopt different methods for different projects. Instead, we recommend that IT organizations should adopt a method-engineering approach that provides them with a dynamic method that can be improved in time and also tailored for different projects when the need arises. Siau (1999) recommends incorporating the consideration of cognitive psychology to add further support for the human element involved in ME.

Method engineering (Brinkkemper, 1996; Brinkkemper, Saeki, & Harmsen, 1999; Kumar & Welke, 1992) is an approach in which methods are first componentized (i.e., we no longer have a single method but a number of method chunks or method fragments; Ter Hofstede & Verhoef, 1997). These method fragments are stored in some kind of library or repository, sometimes known as a methodbase (Ralyté, 1999; Saeki, 2003). Prior to the next project, the method engineer, usually someone on the staff of the organization (perhaps with the help of an outside consultant or a software tool such as a CAME<sup>4</sup> tool; Nguyen & Henderson-Sellers, 2003; Saeki; Tolvanen, Rossi, & Liu, 1996), selects those method fragments needed and the order in which they will be put together. In other words, a method is constructed or engineered from its

component parts in such a way that only those parts relevant are used and those not useful are ignored (e.g., Brinkkemper et al., 1999). Construction rules are required for this assembly process (Ralyté & Rolland, 2001; Tolvanen, 1998). This is arguably one of the most challenging parts of method engineering. Postconditions and preconditions must be carefully matched so that, for instance, a work product required (as part of the precondition) of a method fragment is produced by the preceding method fragment (e.g., Noll, 2003). Initial suggestions have been made by Brinkkemper et al., although no authors, to the best of our knowledge, have yet addressed the question as to how to evaluate the overall situated method for quality and appropriate fit to the organization's methodology requirements. Furthermore, to construct a lightweight or agile process, any highly bureaucratic method fragments, perhaps those involving the writing of copious numbers of reports to management, are ignored; especial care is taken in selecting the particular roles from the repository's collection of role definitions prior to selecting the best people for this development team.

Implicit or explicit in method engineering is the need for an underpinning metamodel (Henderson-Sellers, 2003; Jarke, Pohl, Weidenhaupt, & Lyytinen, 1998; Ralyté &

Rolland, 2001; Tolvanen, 1998). A metamodel describes the concepts used in any method and is the basis for the generation of a number of instances (in the repository) from each of its elements. In other words, for every element carefully and unambiguously defined in the metamodel, there are many examples (instances) stored of this concept in the method repository (Figure 1). For example, for a role concept in the metamodel, we might have roles defined in the repository such as lead developer, tester, prototyper, designer, architect, and so on. As well as the basic process, modeling, and notational fragments, we can also store definitions of the life cycle models in the repository so that the development team can select whether to use a spiral, fountain, contract-driven, or even waterfall life cycle approach. This is in contrast to predefined methods, whether lightweight or heavyweight, which embody a single choice of life cycle model that cannot be changed even if the development team has a different preference or if research and best practice identify an even better or more modern life cycle description.

The use of method engineering has three direct advantages to the software-development organization and to the software-development team:

1. The method that is constructed for the current project is ultimately what is required in terms of processes, activities, tasks, techniques, and guidelines (depending on your choice of terminology) as well as in terms of the people involved (actually roles, as we noted previously), the extent of bureaucratic reporting and project management, and the life cycle model itself.
2. For subsequent projects, the team will develop confidence in self-tuning (tailoring) their own method to fit these projects and also in responding to any change in the development environment. Perhaps the scope of the projects they undertake will change: As they grow in sophistication and software-development capability, so must the process (see also Bajec, Krisper, & Rupnik, 2004). Consequently, elements in the method

repository previously eschewed may now prove useful. Perhaps there is a technique for finding classes and agents that the project team felt they did not have the skills to use previously; perhaps there is a training role for team expansion.

3. Method engineering builds trust in the team members' ability to make use of their method. The development team will be highly motivated and enthusiastic to fully utilize their method as a result of gaining the great feeling of method ownership.

### **Engineering an Agile Method Using the OPEN Process Framework**

To fulfill the requirements for successful method adoption and success in achieving dual agility, we need a methodological approach that supports these ideas of (1) an underpinning metamodel, (2) a component repository for method chunks, and (3) construction guidelines. In other words, we propose a close integration between method engineering, software process improvement, and agile methods.

The ISO12207 and ISO15504 documents fulfill the second criterion, but have no explicitly defined underpinning metamodel, although the OOSPICE project (Stallinger, Dorling, Rout, Henderson-Sellers, & Lefever, 2002; see also <http://www.oospice.com>), which aims to extend the SPICE (ISO15504) standard to CBD, includes an explicit metamodel (Gonzalez-Perez, McBride, & Henderson-Sellers, in press). Research work in method engineering, such as that of Brinkkemper (1996) and Brinkkemper et al. (1999), provides weak support under all three criteria, but this approach appears not to have been applied to any real methodological approach. More recent experimental tool-development work is also reported by Henninger et al. (2002).

At present, one approach that accomplishes these goals in a substantial and commercially usable fashion is that embodied in the OPEN project (<http://www.open.org.au>) and, especially, in the OPEN process framework (Firesmith & Henderson-Sellers, 2002). This work and that of Brinkkemper (1996) and Brinkkemper et al. (1999) start with the method-engineering and repository approach. With the recent adop-

Figure 2. Method engineering with the OPEN Process Framework (after Henderson-Sellers & Hutchinson, 2003)

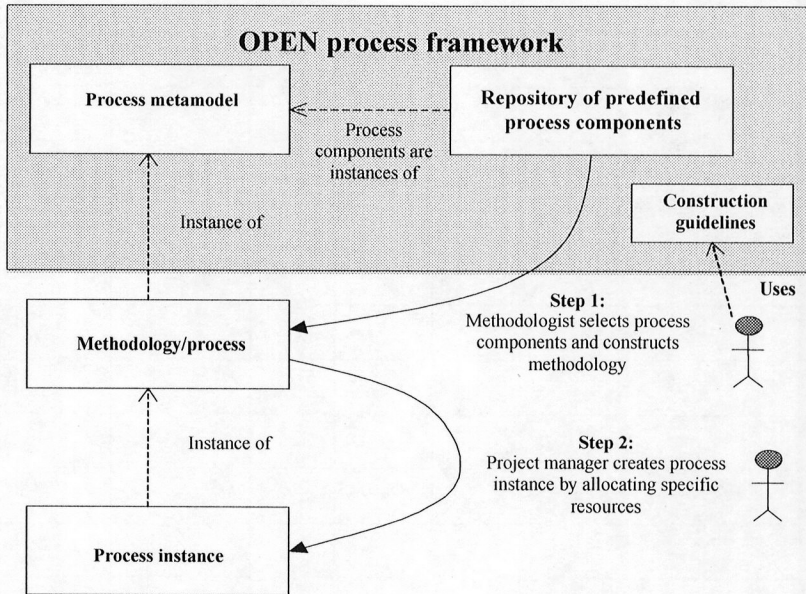
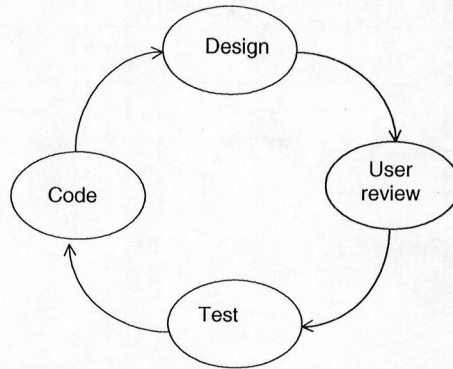


Figure 3. An example of a method appropriate for an agile or lightweight development project, created from the OPEN Process Framework (after Henderson-Sellers, 2002)



tion of the SPEM (software process engineering metamodel) by the OMG (2002), other approaches, originally defined without mention of a metamodel, are in the process of retrofitting metamodels to their methods.

The OPEN process framework is defined by a metamodel that supports the concept of method engineering and process construction (Firesmith & Henderson-Sellers, 2002; Graham, Henderson-Sellers, & Younessi, 1997). OPF provides a rich repository of method components or fragments, which can be used in different soft-

ware projects. OPF also provides a set of guidelines that offer advice on the selection of specific method components based on the notion of possibility matrices linking each pair of method fragments (Henderson-Sellers, Haire, & Lowe, 2002). Figure 2 illustrates the OPF fundamentals and the mechanism of method construction using the method-engineering approach.

In OPF's process metamodel, there are elements to describe process components such as activities, tasks, and techniques; people components such as producers and roles; organi-

Figure 4. Representative elements of the activity/task matrix for an agile or lightweight method

| Task                                                | Activity |      |      |             |
|-----------------------------------------------------|----------|------|------|-------------|
|                                                     | Design   | Code | Test | User review |
| Identify project roles and responsibilities         | Y        | Y    | Y    | Y           |
| Analyze user requirements                           | Y        |      | Y    |             |
| Identify classes, objects, roles, types, interfaces | Y        |      |      |             |
| Design the human interface                          | Y        |      |      | Y           |
| Refactor                                            | Y        |      |      |             |
| Integrate components                                | Y        | Y    | Y    |             |
| Code                                                |          | Y    |      |             |
| Plan testing strategy                               |          |      | Y    |             |
| Design test suite                                   |          |      | Y    |             |
| Code test suite                                     |          | Y    | Y    |             |
| Execute tests                                       |          |      | Y    |             |
| Report on test results                              |          |      | Y    |             |
| Deliver product to users                            |          |      |      | Y           |
| Train users                                         |          |      |      | Y           |
| Identify appropriate reusable work products         |          |      | Y    |             |
| Evaluate quality                                    |          |      | Y    |             |
| Evaluate the design                                 |          |      | Y    |             |

zational components, such as the enterprise, program, and project; and product components in the form of a whole range of work products including diagrams and documents, supported by various kinds of languages (natural language, modeling language, and coding language). For a dual-agility process, at the top level, we select only a small number of the activities together with a life cycle model. For instance, a typical iterative life cycle model with only four major activities, perhaps something akin to XP (Beck, 2000), could be depicted graphically as shown in Figure 3.

The next stage is to realize the selected activities in terms of the tasks and techniques that will enable the developers to do the job. So, for each of the four activities of Figure 3, we next identify appropriate low-level tasks that the developers must accomplish. This is done using an important and unique element of OPEN: its deontic matrices (Figures 4 and 5). There is a many-to-many relationship between various pairs of meta-elements in the metamodel. For any chosen activity, there are often choices to be made between appropriate tasks (Figure 4), and similarly between the chosen tasks and

Figure 5. A portion of the task/technique matrix for an agile or lightweight method

| Technique                                     | Tasks |   |   |   |   |   |   |
|-----------------------------------------------|-------|---|---|---|---|---|---|
|                                               | 1     | 2 | 3 | 4 | 5 | 6 | 7 |
| Beta testing                                  |       |   |   | Y | Y |   | Y |
| Regression testing                            |       |   |   | Y | Y |   |   |
| Abstract class                                | Y     |   |   |   |   |   |   |
| Generalization and inheritance identification | Y     | Y |   |   |   |   |   |
| Pattern                                       | Y     | Y | Y |   |   | Y |   |
| Dialogue design in UI                         |       | Y |   |   |   |   |   |
| Usability testing                             |       | Y |   | Y | Y |   |   |
| Class internal design                         |       |   | Y |   |   |   |   |
| Implementation of services                    |       |   | Y |   |   |   |   |
| Implementation of structure                   |       |   | Y |   |   |   |   |
| Genericness specification                     | Y     |   |   |   |   |   |   |
| Revision of inheritance hierarchies           |       |   |   |   |   | Y | Y |
| Class/type indexing                           |       |   |   |   |   |   | Y |
| Library class incorporation                   |       |   |   |   |   | Y |   |

**KEY:**

1. Identify classes, objects, roles, types, interfaces
2. Design the human interface
3. Code
4. Design test suite
5. Execute tests
6. Identify appropriate reusable work products
7. Evaluate the design

the most appropriate techniques (Figure 5). In the latter case, for example, Figure 4 identifies that the task, designing the human interface, may be effected by four selected techniques (labeled Y), yet the technique, implementing services, might have been chosen but was not (matrix element left blank as here or labeled N). While OPEN prescribes five levels of possibility in this matrix, here we use only two (to use or do not use) since we find two is adequate for an initial method construction. Five is useful either in describing methods appropriate for whole domains or for individual projects once the team is more gelled and more sophisticated. For an agile project, this and the other matrices (e.g., between task and technique, and between task and work product) will necessarily be sparse; that is, they will be mostly blank or in combinations that are not useful. Roles and work products can be identified easily and allocated in a similar way.

Finally, it must be stressed that method engineering does not provide agility per se. As

noted earlier, Cockburn (2003) identifies agility with the existence of a mind-set and not with any particular techniques. The OPF currently does not have a metaclass called a mind-set. Although such a philosophically focused metaclass might be considered for future incorporation into the OPF metamodel, for the present, we note that an XP style is embodied in the notion of the metaclass called the life cycle model plus the repository-held process components of (1) the XP life cycle model and (2) the XP-style team.

## EMPIRICAL EVALUATIONS

In order to evaluate empirically the value of agility and dual agility, two major action-research (AR) studies were undertaken with Sydney-based organizations. The focus of the research was the testing of the hypothesis that the construction and utilization of an organization-specific dual-agility method may enhance the overall performance of the software-development team due to its flexibility and

customizability to accommodating different projects. That this is facilitated by the use of method engineering is a valuable, yet secondary, evaluation based on the introduction of the OPEN process framework from the repository of which can be constructed an organizationally specific dual-agile method, which can be further tailored and customized to be optimal for each individual project. Software projects, even at the organizational level, are expected to be very different in domain, scope, size, complexity, and type. Thus, selecting one single method for all these different software projects seems impractical and infeasible (Serour & Henderson-Sellers, 2004a), and the use of SME vital. It is critical that the method fits the business and not the other way around.

Since many current software projects change very frequently to the extent that the deployed method must be responsive enough to respond to any changes promptly and effectively, a dual-agility attribute for the software-development method becomes almost inevitable. The fast-changing development environment and more specific user requirements demand more flexibility from processes in order to satisfy the demands created and to achieve process improvement through a continual introspection as well as the use of retrospectives (Kerth, 2001).

### **Empirical Evaluation and Action Research**

Rather than assisting our study organizations in a consultancy mode, we persuaded them to be part of an ongoing research project using the action-research methodology. Action research is a research methodology originally used in education research and more recently in information-systems research (Avison, Lau, Myers, & Nielsen, 1999). Using action research, the researcher proposes a hypothesis and then tests it in an industrial setting, acting as both external observer and internal team member. AR was chosen by our research team since AR offers the ability of the researcher(s) to contribute both to the practical concerns of people in their immediate problematic situation and to the goals of social science by joint collaboration within a mutually acceptable ethical framework

(Argyris, Putnam, & Smith, 1985; Hult & Lennung, 1980; Lau, 1999; Lewin, 1974; Rapoport, 1970; Reason, 1993; Susman & Evered, 1978). The action-research methodology focuses on the collaboration between researchers who are aiming to test and/or prove their theory and practitioners who are aiming to solve their immediate problem(s) and/or enhance their current work culture (Avison et al., 1999). In other words, AR has the dual aims of providing a mechanism for practical problem solving (action), and for testing and enhancing theory (research). Elden and Chisholm (1993) argue that the dual interest of AR provides a win-win scenario for both researcher and participants, and plays an effective role in solving practical problems by increasing the understanding of a given social situation through the direct involvement of the researcher in an organizational change that can also positively affect future decisions and actions based on better understanding of the problem(s) at hand (Avison et al.; Hult & Lennung). As a result, action research, as an effective qualitative research method, has been widely adopted and utilized for studies in many different disciplines.

### **Empirical Evaluation for Dual-Agility Methods**

We undertook two action-research experiments with organizations employing an agile process constructed using an SME approach. The first project (Serour & Henderson-Sellers, 2004b) took more than 2 years and successfully assisted an IT department within a legal publishing company in Sydney in constructing an agile method for its new Web-development projects. The second project (Serour & Henderson-Sellers, 2004a) was conducted for more than a year in a governmental department within the New South Wales (NSW) government in Australia during its transformation process to e-government. This latter project successfully empowered the IT section within the governmental department with a new constructed agile method, the usage-centered design (UCD) approach, and a core set of the unified modeling language (UML) for their e-gov-

*Table 5. Results of the SPICE assessment undertaken prior to commencement of the two pilot projects*

| Process to be assessed   | SPICE level achieved |
|--------------------------|----------------------|
| Requirements elicitation | 1                    |
| Software development     | 1                    |
| Configuration management | 0                    |
| Quality assurance        | 0                    |
| Problem resolution       | 0                    |
| Project management       | 1                    |
| Risk management          | 0                    |
| Process establishment    | 0                    |

ernment Web-development initiative (Serour & Henderson-Sellers, 2004c, 2004d).

The main objective of these empirical evaluations was to evaluate the introduction of a method-engineering approach using the OPF in order to test our construction of a dual-agile method for these two organizations: a construction done in such a way that the methodology can be fully customized to suit individual projects.

### **First Project Background**

The first organization we studied is a well-established, multinational legal publishing company and a leading publisher of legal and tax information to the professional-services industries. In the past, the company's focus had been on the production and distribution of printed material and compact discs. Six years ago, a decision was made to transform the organization into a provider of online information in its domain, requiring a transformation of their traditional business practices to an electronic business environment (e-business). The IT department within the organization was in a desperate need for a new approach to manage and control its software projects by deploying a method that would be flexible and agile enough to accommodate the requirements of its new software-development environment. Moreover, the IT department's customers

(other departments within the organization) required that software systems and the organization's Web sites must be able to adapt more easily to changing business needs, must meet their unique requirements, and must scale across large and small installations.

### *Assessing the Existing IT Maturity Level*

Before the commencement of this project, the IT division of the study organization was assessed in terms of its existing software-development process by a qualified third-party consulting firm using the SPICE model with its rapid organization-assessment technique. Software-development maturity was assessed as being roughly equivalent to Level 1 of the CMM scale (Table 5). The organization declared an aim of achieving Level 2 or 3 of the SPICE scale (equivalent to CMM Level 3) within a relatively short time frame, as well as of providing a better online product than its competitors. By adopting or constructing an initial agile method and later enhancing it (by adding the second degree of agility incrementally), for example, by including metrics collection, they are well on the way to a successful transition to a higher CMM level within their selected time frame (Serour & Henderson-Sellers, 2004d).

### *Evaluating the Existing Methods*

Initially, management decided to conduct a formal evaluation of the contemporary OO software-development processes and methodologies to choose the most appropriate one to be adopted by its staff. Management insisted and encouraged everyone to participate in that evaluation whether by attending the evaluation sessions or by providing suggestions and feedback. Based on its analysis of the final evaluation report, management concluded that selecting one single method or process for all its different software projects, even at the organizational level, seemed impractical and infeasible. As a result, the organization, with everyone's agreement, decided to adopt a method-engineering approach instead of adopting a predefined method to construct its own method that can be tailored to suit its individual projects. After conducting a number of education sessions to introduce the organization to the concepts of method engineering and the contemporary process frameworks, management chose OPF to be utilized to construct the new method. One of the major reasons for choosing OPF was due to its unique characteristics of customizability, tailorability, and flexibility. These features allow IT people within the study organization to create their own method in an incremental manner following the concept of "small wins" (Serour, Henderson-Sellers, Hughes, Winder, & Chow, 2002). This was emphasized by the IT director, who said, "We had selected the OPF as our preferred process framework due to its tailorability, its alignment with object-oriented analysis and design and its explicit support for iterative development and time boxing, crucial characteristics in a rapid delivery environment such as ours."

### *Project Planning*

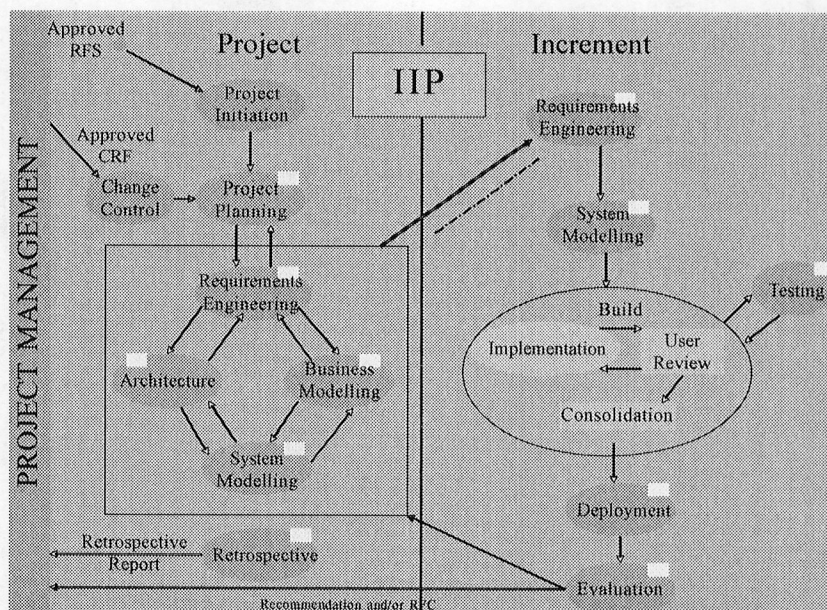
Management formed a new team called the method-engineering team from a number of IT people, two customer representatives, and the organization IT director to achieve some imperative goals including:

- Giving IT people the feeling of the ownership of their methodology,
- Ensuring customer involvement, and
- Representing management commitment.

In this experiment, a minimal subset of method fragments was chosen purposefully from the OPF method repository. The Web-development team within the study organization was facing an increased pressure to develop a set of new Web-based applications under evolving requirements and changing technologies within a very tight time frame. The method-engineering team realized the urgent need for engineering a process that is more adaptive than predictive, and is more people oriented than process oriented: The newly constructed agile process was thus in accordance with the Agile Manifesto as exemplified by the four key values of the "Manifesto for Agile Software Development" (Table 1). This method was only successful for the very first project (Serour et al., 2002). For the second and subsequent projects, we identified some existing method fragments as being unnecessary and other method fragments that needed to be added. As a result, we had to change the company's previously engineered method, thus adding the second agility feature by supplying an ability for the developer to self-tune or reengineer the method to accommodate different projects. For nearly eight months, both the researcher and practitioner teams worked in close collaboration with everyone involved, including customers, to produce a flexible method to satisfy the requirements of each and every project of the organization. Finally, the organizational dual-agility method was created along with a number of easy-to-read documents. It was well accepted and is still cherished by both management and technical people. Figure 6 illustrates the software-development method that was engineered for the organization.

As a result, on second and subsequent projects, more advanced tasks and techniques were added to help the company to "grow" its method, thus extending its scope and skills level.

Figure 6. The engineered method for the study organization



## Second Project Background

The second organization studied is a well-established governmental department within the NSW state government of Australia that provides public services to more than 4 million customers in the communities of Sydney and other cities. It has around 3,500 full-time staff members in general and about 120 personnel in the IT section headed by the IT manager and made up of a mixture of business and system analysts, developers, testers, and customer supporters. The main focus of the IT section is developing and maintaining software applications for other sections within the department including human resources, product management, and product monitoring. Some years ago, a decision was made to provide the community with some online services such as paying bills, submitting general enquires, and viewing public reports. The compelling reason for this decision is the need for the department to remain competitive in the online public-services arena. A new IT manager was appointed to lead the IT

organization through its endeavor to change the culture, and upgrade and enhance its capabilities to be able to provide effective online products for ultimate customer services.

## First Adoption Attempt

Prior to the authors' involvement with this project, senior management, influenced by the IT manager, made a decision to adopt a pre-defined, formal, and heavyweight OO software-development methodology along with a recognized modeling language and CASE tool to develop software applications for the new online service environment. The selected pilot project was the development of major applications for a customer-management system due to its vital role in providing ultimate online customer services to the public. This attempt was welcomed by top senior management and strongly resisted by most IT developers, with there being two main reasons behind their resistance. First, the proposed software method was too large to learn and comprehend over a short period of time

and was soon identified by the development team as being much too heavyweight for transitioning the IT personnel to a new software-development culture. Second, the selected pilot project was very large in size and critical in nature, requiring special skills and expertise for its implementation.

Despite other attempts to outsource the major customer-support applications, senior management realized the problem and decided to terminate the project and look for a more effective, feasible, and practicable solution to their immediate problem.

From a research point of view, we can identify several factors that are likely to have had a negative impact on this first adoption attempt:

- The inappropriate introduction of the new software-development method with its associated techniques and tools, which strongly increased people's resistance to change,
- The lack of people's participation during the stage of assessing and selecting the most appropriate method for the organization's needs, which negatively related to people's feeling of ownership and the sharing of values with their organization,
- The existence of a communication gap between top management and IT personnel, and
- The lack of adequate and appropriate education and training that people needed in order to feel confident about adopting the new approach.

Not surprisingly, these factors contributed significantly toward people's attitude against the new changes and led directly to the hesitance of the first attempt.

### *Second Adoption Attempt with Method Engineering*

Following a number of discussions with our research team, the senior IT manager and the Web-development manager realized the immediate need for changing their existing work

culture and adopting and/or constructing a new method that would satisfy and fulfill the requirements of their new software-development environment. Consequently, they decided to initiate a further attempt to change through the adoption of a method-engineering approach to construct a new agile method, and through the adoption of other techniques and tools including the usage-centered design approach (Constantine & Lockwood, 1999) and a core set of the UML modeling language (OMG, 2001).

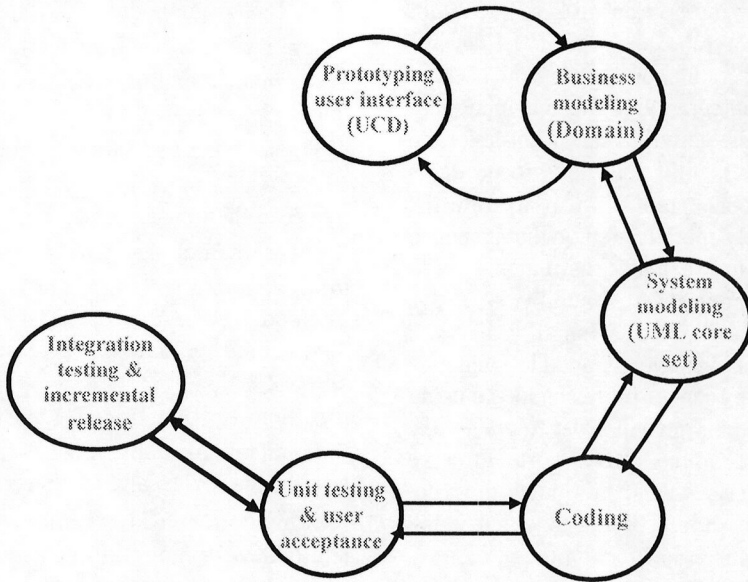
Thus, the major objective of this second project was empowering the software-development team with new technologies for Web development to be able to support the new work environment. The IT section within the government department had to undergo a transition process in order to change its existing work culture, and adopt new technologies for Web development in order to be sufficiently competent to support the new e-government business processes. These new technologies were the following (see Serour & Henderson-Sellers, 2004a, for further details):

- The construction of a specific dual-agile software-development method using the method-engineering approach and the OPF in an incremental and iterative manner,
- The adoption and utilization of the UCD approach for designing the user interface of the new e-government applications, and
- The adoption and utilization of a specific core set of the UML for modeling the new e-government applications.

### *The Transition Process with Small Wins*

The whole transition process started by transitioning a small team (the online team was chosen to support the e-government transformation) with the anticipation of extending this later to other teams once initial success had been gained. As well as being more manageable, a small team has a higher chance of success: That success then positively influences other teams. We call this technique small wins (see Serour, 2003; Serour et al., 2002): a tech-

Figure 7. The constructed IT method to support e-government OPEN, usage-centered design, and a UML core set (after Serour & Henderson-Sellers, 2004)



nique that has proven to be very effective in changing people's culture and managing their natural resistance.

Following the principles of the small-wins technique, the proposed new technologies were introduced and the culture change process was initiated in an incremental, iterative, and parallel manner. The construction of the new agile method was started by addressing the requirements-engineering (RE) activity as a major focus to engender everyone's involvement. An initial RE approach, using the use-case technique, was introduced, as this was nonexistent in the existing culture, to emphasize and achieve the following objectives:

- Gather the required information for the project effectively,
- Document the immediate requirements using a template to maintain a standard,
- Receive customer input using the requirements documentation as reference, and
- Run the appropriate tests and requirement tractability before every release.

This new requirement-engineering approach was carried out for two pilot projects

and proved beneficial to both. The notion of "only use what you need" allowed this approach to be accepted and adopted without major resistance from the development team. With this project, the customer was directly involved in checking the use-case documents, noting any changes, and signing off this particular project for development. The customer sign-off at this stage was an innovation and was deemed beneficial by both the customers and developers.

Toward the end of the first year, the online team in collaboration with the research team created the first version of the department's method as shown in Figure 7. The new organizational method was successfully constructed using a method-engineering approach with the utilization of the OPEN process framework.

The new method was totally designed according to the organizational requirements and fully engineered to suit the new IT work environment. As a result, the new method was fully accepted and utilized by all IT personnel with almost no resistance to change their existing work culture. As requested by the IT personnel, the new method was designed to promote and support the concepts of iteration, in-

crements, and parallelism to best manage the task of producing software systems.

The new method consists of six major activities: prototyping, business modeling, system modeling, coding, unit testing and user acceptance, and integration testing and incremental release. These activities state high-level and large-scale work units that need to be done, and they are connected in a way to form the organizationally specific method that is seen in the work flow between these activities. In turn, each activity consists of a number of tasks. Each task can be defined as a small-scale work unit that models a single assigned job that may be performed by one or more producers using one or more techniques in order to create one or more work products. It is important to note that a task describes what a producer must do, but not how it should be done, which is the realm of the appropriate technique(s).

In addition, the online team has successfully adopted the UCD approach (Constantine & Lockwood, 1999) as amalgamated with OPEN (Henderson-Sellers & Hutchison, 2003) for designing the user interface that was efficiently used in the first activity to design and produce effectual user interfaces. Moreover, it selected an appropriate subset of the UML that was fully utilized during the activities of system and business modeling. These achievements have empowered the IT section within the government department with new technologies and approaches that enabled it to support the entire organization during its transformation process to e-government.

### **Empirical Project Findings and Achievements**

The method-engineering and dual-agility approaches were strongly recommended to both study organizations for the following reasons:

- Each organization must develop its own way of working contrary to the adaptation to an existing way (Cockburn, 2002);
- Based on previous experience, the idea of adopting an existing method, even one that

was agile, was rejected due to people's belief that they must adapt themselves to suit the adopted method;

- There was the need for a method that provides maneuverability to deal with requirement changes;
- There was the need for a method that enables teams to deliver software products faster to their customers;
- Dual agility not only provides a better way of developing software, but it can be enhanced by the use of SME to support method customization to best suit people's needs;
- For both organizations, IT customers are internal and available most of the time for immediate review and feedback; and
- Getting customer involvement through the entire development life cycle is a top priority as in the Agile Manifesto.

Experience gained during these action-research projects showed that the second agility feature is as important as the first agility feature. During both of these empirical studies, with the private-sector legal publishing company and the public-sector governmental department in Sydney, we assisted the IT organization (or development team) in creating an effective agile method to be used by the organization for its immediate Web development.

The process of method engineering, with a rich repository of process fragments such as the OPF, has changed everyone's perception of the main role of a software-development method, which, in turn, was a driving force for culture change and resistance management. It was clearly noticed through our projects that everyone was convinced that the new method neither measured people's productivity nor tracked their performance. Rather, the deployment of a dual-agility method with associated techniques, such as UCD and UML, could be a new way of not only delivering reliable software products to customers on time and within budget, but of continually enhancing the maturity level of the software-engineering team.

Another observed advantage of method engineering was the new people's feeling that the method is an organizational property and does not belong to any individual or a group of individuals within the organization; rather, it is owned by everyone. The ubiquitous feeling of ownership considerably helped management in both organizations in eliminating people's resistance to change. They have shown their willingness and readiness to participate and support their transition to the new software-development environment with no fear and less uncertainty.

### Management's Acknowledgment of Success

The following quotes were articulated by senior management toward the end of our two empirical projects as a means of expressing their feeling toward the entire change process to a new work environment.

"The research team has succeeded in leading us to the path of biting off the transition in small, self-sustaining chunks rather than a 'big bang' approach and ensuring that all parties had the opportunity to participate and influence the outcome" says the change management team leader of the first project

"A follow-up benchmark by an independent external organisation established that the processes as defined would achieve CMM 2 once the requisite experience was achieved to embed it into the organisation," says the IT director of the first project.

"As a result we were able to achieve significant improvement in the predictability of project outcomes and enhance our ability to introduce new project members effectively," says the general development manager of the second project.

### SUMMARY AND CONCLUSION

Agile methods make an important contribution to the practice of software development. However, the plethora of currently available methods means that developers must commit to one particular method brand and are thus prevented in the future from making drastic

changes in their methodological approach while remaining committed to their first choice. Some path is needed to allow such developers, as they mature organizationally, to migrate to a more appropriate method as contexts change.

Here, some ideas on method engineering have been presented in the context of the construction of an agile method using the metamodel of the OPF. Selecting method fragments from the OPF method repository supports the construction of a minimalistic agile method, perhaps paralleling XP or Scrum, but then also further supports the migration of the method itself to more sophistication if and when it is found necessary. This not only supports the well-known flexibility within an agile method, but also agility in the lifetime of the method package itself (dual agility). Thus, developers can operate within a flexible and agile environment while at the same time maintaining the opportunity for future SPI. This double flexibility underscores the value of a method-engineering approach in the creation of a truly and fully agile method.

### ACKNOWLEDGMENTS

We wish to thank Cesar Gonzalez-Perez and Darryl Winder for providing useful feedback and comments on an early draft of this article. This is Contribution Number 03/06 of the Centre for Object Technology Applications and Research (COTAR).

### REFERENCES

- Abrahamsson, P., Warsta, J., Siponen, M. T., & Ronkainen, J. (2003). New directions on agile methods: A comparative analysis. In *Proceedings of ICSE '03*, (pp. 244-254). Los Alamitos, CA: IEEE Computer Society Press.
- Ambler, S. (2003). *Agile database techniques*. J. Wiley & Sons.
- Argyris, C., Putnam, R., & Smith, D. M. (1985). *Action science: Concepts, methods and skills for research and investigation*. San Francisco, CA: Jossey-Bass Publisher.
- Avison, D. E., Lau, F., Myers, M., & Nielsen, P. A. (1999). Making academic research more relevant. *Communications of the ACM*, 42(1), 94-97.

- Aydin, M. N., Harmsen, F., van Slooten, K., & Stegwee, R. A. (2004). Appropriate delivery of advice and guidance on method adaptation. In *Proceedings of AMCIS 2004*.
- Bajec, M., Krisper, M., & Rupnik, R. (2004). The scenario for constructing flexible, people-focused systems development methodologies. In *Proceedings of ECIS 2004*.
- Beck, K. (2000). *Extreme programming explained*. Addison-Wesley.
- Beck, K., & Boehm, B. (2003). Agility through discipline: A debate. *IEEE Computer*, 36(6), 44-46.
- Bettin, J. (2003). Practicalities of implementing component-based development and model-driven architecture. In *Process Engineering for Object-Oriented and Component-Based Development: Proceedings of OOPSLA 2003 Workshop*, Sydney, Australia (pp. 19-30).
- Brinkkemper, S. (1996). Method engineering: Engineering of information systems development methods and tools. *Information Software Technology*, 38(4), 275-280.
- Brinkkemper, S., Saeki, M., & Harmsen, F. (1999). Meta-modelling based assembly techniques for situational method engineering. *Information Systems*, 24(3), 209-228.
- Chau, T., Maurer, F., & Melnik, G. (2003). Knowledge sharing: Agile methods vs. Tayloristic methods. In *Proceedings of the 12th IEEE International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE'03)*, (pp. 302-307). Los Alamitos, CA: IEEE Computer Society Press.
- Cockburn, A. (2000). Selecting a project's methodology. *IEEE Software*, 17(4), 64-71.
- Cockburn, A. (2002). An interview with Alistair. *The Cutter Consortium*. Retrieved on February 23, 2004, from <http://www.cutter.com/consultants/cockburna.html>
- Cockburn, A. (2003). *The Cutter Consortium*. Retrieved April 7, 2003, from <http://www.cutter.com/consultants/cockburna.html>
- Constantine, L. (2002, August-September). Non-skid agility? *Information Age*, 34-42.
- Constantine, L. L., & Lockwood, L. A. D. (1994). One size does not fit all: Fitting practices to people. *American Programmer*, 7(12), 30-38.
- Constantine, L. L., & Lockwood, L. A. D. (1999). *Software for use*. Addison-Wesley.
- Elden, M., & Chisholm, R. F. (1993). Emerging varieties of action research: Introduction to the special issue. *Human Relations*, 46(2), 121-142.
- Firesmith & Henderson-Sellers. (2002). *The OPEN process framework: An introduction*. Addison-Wesley.
- Fowler, M. (2002). *The new methodology*. Retrieved from <http://www.martinfowler.com/articles/newMethodology.html>
- Gonzalez-Perez, C., McBride, T., & Henderson-Sellers, B. (in press). A metamodel for assessable software development methodologies. *Software Quality Journal*.
- Graham, I., Henderson-Sellers, B., & Younessi, H. (1997). *The OPEN process specification*. Addison-Wesley.
- Henderson-Sellers. (2002). Agile or rigorous OO methodologies: Getting the best of both worlds. *Cutter IT Journal*, 15(1), 25-33.
- Henderson-Sellers, B. (2003). Method engineering for OO system development. *Communications of the ACM*, 46(10), 73-78.
- Henderson-Sellers, B., Haire, B., & Lowe, D. (2002). Using OPEN's deontic matrices for e-business. In C. Rolland, S. Brinkkemper, & M. Saeki (Eds.), *Engineering information systems in the Internet context* (pp. 9-30). Boston: Kluwer Academic Publishers.
- Henderson-Sellers, B., & Hutchison, J. (2003). Usage-centered design (UCD) and the OPEN process framework (OPF). In L.L. Constantine (Ed.), *Performance by Design: Proceedings for USE2003, Second International Conference on Usage-Centered Design*, (pp. 171-196). Rowley, MA: Ampersand Press.
- Henninger, S., Ivaturi, A., Nuli, K., & Thirunavukkaras, A. (2002). Supporting adaptable methodologies to meet evolving project needs. In *Proceedings of the First ICSE Workshop on Iterative, Adaptive, and Agile Processes*, Orlando, FL., May 25.
- Highsmith, J. A. (2000). *Adaptive software development: A collaboration approach to managing complex systems*. Dorset House.
- Hult, M., & Lennung, S. (1980). Towards a definition of action research: A note and bibli-

- ography. *Journal of Management Studies*, 17(2), 241-250.
- Jalote, P. (2002). Lessons learned in framework-based software process improvement. In *Proceedings of the Ninth Asia-Pacific Conference on Software Engineering, APSEC2002*, (pp. 261-268). Los Alamitos, CA: IEEE Computer Society Press.
- Jarke, M., Pohl, K., Weidenhaupt, K., & Lyytinen, K. (1998). Meta modeling: A formal basis for interoperability and adaptability. In B. Krämer & M. Papazoglou (Eds.), *Information systems interoperability* (pp. 229-263). John Wiley Research Science Press.
- Kerievsky, J. (2001). Continuous learning. In *Proceedings of XP2001*. Retrieved from <http://www.xp2003.org/conference/papers/Chapter1-Kerievsky.pdf>
- Kerth, N. L. (2001). *Project retrospectives: A handbook for team reviews*. New York: Dorset House.
- Kumar, K., & Welke, R. J. (1992). Methodology engineering: A proposal for situation-specific methodology construction. In W. W. Cotterman & J. A. Senn (Eds.), *Challenges and strategies for research in systems development* (pp. 257-269). Chichester: J. Wiley.
- Lau, F. (1999). Toward a framework for action research in information systems studies. *Information Technology and People*, 12(2), 148-175.
- Lewin, K. (1974). Frontiers in group dynamics I. Concepts, method and reality in social sciences: Social equilibria and social change. *Human Relations*, 1, 5-41.
- Lindvall, M. et al. (2002). Empirical findings in agile methods. In *Extreme Programming and Agile Methods, XP/Agile Universe 2002: Proceedings of the Second XP Universe and First Agile Universe Conference*, 197-207.
- Lyytinen, K. et al. (1994). *MetaPHOR: Metamodeling, principles, hypertext, objects and repositories* (technical report no. TR-7). Finland: University of Jyväskylä, Department of Computer Science and Information Systems.
- MacIsaac, B. (2003). An overview of the RUP as a process engineering platform. In *Process Engineering for Object-Oriented and Component-Based Development: Proceedings of OOPSLA 2003 Workshop*, (pp. 43-52).
- Newkirk, J. (2002). Introduction to agile processes and extreme programming. In *Proceedings of ICSE '02*, (pp. 695-696).
- Nguyen, V. P., & Henderson-Sellers, B. (2003). Towards automated support for method engineering with the OPEN approach. In *Proceedings of the Seventh IASTED International Conference on Software Engineering and Applications*, (pp. 691-696).
- Noll, J. (2003). Flexible process enactment using low-fidelity models. In *Proceedings of the Seventh IASTED International Conference on Software Engineering and Applications*, (pp. 675-680). Anaheim, CA: ACTA Press.
- OMG. (2001). *OMG unified modelling language specification, version 1.4* (OMG documents formal/01-09-68 through 80). Retrieved on July 12, 2002, from <http://www.omg.org>
- OMG. (2002). *Software process engineering metamodel specification, version 1.0* (OMG document formal/02-11-14).
- Opperthausen, D. (2003, September). Defect management in an agile development environment. *Crosstalk Journal of Defense Software Engineering*.
- Rainer, A., & Hall, T. (2002). Key success factors for implementing software process improvement: A maturity-based analysis. *Journal of Systems and Software*, 62(2), 71-84.
- Ralyté, J. (1999). Reusing scenario based approaches in requirement engineering methods: CREWS method base. In *Proceedings of the 10th International Workshop on Database and Expert Systems Applications (DEXA '99) and the First International Workshop on the Requirements Engineering Process: Innovative Techniques, Models, Tools to Support the RE Process (REP '99)*, Florence, Italy, September 1-3 (pp. 305-309). Los Alamitos, CA: IEEE Computer Society Press.
- Ralyté, J., & Rolland, C. (2001). An assembly process model for method engineering. In *Proceedings of CAiSE 2001*, (pp. 267-283). Berlin: Springer-Verlag.
- Rapoport, R. (1970). Three dilemmas in action research. *Human Relations*, 23(4), 499-513.
- Reason, P. (1993). Three approaches to partici-

- pative inquiry. In N. Denzin & Y. Lincoln (Eds.), *Handbook of qualitative research* (pp. 324-339). London: Sage Publications.
- Reifer, D. J., Maurer, F., & Erdogmus, H. (2003). Scaling agile methods. *IEEE Software*, 20(4), 12-14.
- Rolland, C., Prakash, N., & Benjamin, A. (1999). A multi-model view of process modelling. *Requirements Engineering Journal*, 4(4), 169-187.
- Saeki, M. (2003). CAME: The first step to automated software engineering. In *Process Engineering for Object-Oriented and Component-Based Development: Proceedings of OOPSLA 2003 Workshop*, Sydney, Australia (pp. 7-18).
- Serour, M. K. (2003). *The effect of intra-organisational factors on the organisational transition to object technology*. Doctoral dissertation, University of Technology, Sydney, Australia.
- Serour, M. K., & Henderson-Sellers, B. (2004a). Empowering a software development team with a new methodology: A case study of e-government in Australia. In *Proceedings of IBIMA*, Amman, Jordan, July 4-6.
- Serour, M. K., & Henderson-Sellers, B. (2004b). Introducing agility: A case study of situational method engineering using the OPEN process framework. In *Proceedings of the 28th Annual International Computer Software and Applications Conference: COMPSAC 2004*, (pp. 50-55). Los Alamitos, CA: IEEE Computer Society Press.
- Serour, M. K., & Henderson-Sellers, B. (2004c). OPEN for agility: An action research study of introducing method engineering into a government sector. In O. Vasilecas, A. Caplinskas, W. Wojtkowski, J. Zupanic, & S. Wrycza (Eds.), *Proceedings of the 13th International Conference on Information Systems Development: Advances in Theory, Practice and Education*, Vilnius, Lithuania (pp. 105-116).
- Serour, M. K., & Henderson-Sellers, B. (2004d). Organizational aspects of transformation to e-business: A case study. In P. Isaías, P. Kommers, & M. McPherson (Eds.), *Proceedings of IADIS International Conference e-Society 2004*, 2, (pp. 751-758). IADIS Press.
- Serour, M., Henderson-Sellers, B., Hughes, J., Winder, D., & Chow, L. (2002). Organizational transition to object technology: Theory and practice. In Z. Bellahsene, D. Patel, & C. Rolland (Eds.), *Object-oriented information systems* (pp. 229-241). Berlin, Germany: Springer-Verlag.
- Siau, K. (1999). Information modeling and method engineering: A psychological perspective. *Journal of Database Management*, 10(4), 44-50.
- Skowronski, V. (2004). Do agile methods marginalize problem solvers? *IEEE Computer*, 37(10), 118-120.
- Stallinger, F., Dorling, A., Rout, T., Henderson-Sellers, B., & Lefever, B. (2002). Software process improvement for component-based software engineering: An introduction to the OOSPICE project. In *Proceedings of Euromicro 2002 Conference*, (pp. 318-323). Los Alamitos, CA: IEEE Computer Society Press.
- Susman, G. I., & Evered, R. D. (1978). An assessment of the scientific merits of action research. *Administrative Science Quarterly*, 23(4), 582-603.
- Taylor, F. (1998). *The principles of scientific management*. Dover Publications.
- Ter Hofstede, A. H. M., & Verhoef, T. F. (1997). On the feasibility of situational method engineering. *Information Systems*, 22, 401-422.
- Tolvanen, J.-P. (1998). *Incremental method engineering with modeling tools: Theoretical principles and empirical evidence*. Doctoral dissertation, University of Jyväskylä, Finland.
- Tolvanen, J.-P., Rossi, M., & Liu, H. (1996). Method engineering: Current research directions and implications for future research. In S. Brinkkemper, K. Lyytinen, & R. Welke (Eds.), *Method engineering, principles of method construction and support* (pp. 296-317). London: Chapman-Hall.
- Turk, D., France, R., & Rumpe, B. (in press). Assumptions underlying agile software development processes. *Journal of Database Management*, 16(4).

## ENDNOTES

<sup>1</sup> Fowler uses these terms synonymously.

<sup>2</sup> Without wishing to get involved in pedantic arguments of terminology, as a working definition, we will use the word "method" to describe the largest scale, which contains two components: a process plus product-focused issues (such as modeling and notational aspects; Rolland, Prakash, & Benjamin, 1999).

<sup>3</sup> In SPICE and OOSPICE terminology, the word "process" refers to any activity that is identifiable as a whole with prescribed inputs and outputs. In any software-development approach, there are likely to be a couple dozens of these processes.

<sup>4</sup> CAME stands for computer-assisted method engineering. A significant research project in the area of ME and CAME was the MetaPHOR project (e.g., Lyytinen et al., 1994; Tolvanen, 1998).

*Brian Henderson-Sellers is director of the Centre for Object Technology Applications and Research and professor of information systems at University of Technology, Sydney (UTS). He is author of eight books on environmental simulation, 12 books on object and agent technology, and is well-known for his work in OO methodologies (MOSES, COMMA, OPEN, OOSPICE) and in OO metrics. He has also contributed to several standards, both in the Object Management Group and Standards Australia and is currently leading work on a new ISO standard. In July 2001, Professor Henderson-Sellers was awarded a doctor of science (DSc) from the University of London for his research contributions in object-oriented methodologies.*

*Magdy Serour is a research fellow at the Centre for Object Technology Applications and Research (COTAR) at the University of Technology, Sydney (UTS). He is the co-founder of SB the Software Group Pty Ltd (1982) and has had 28 years experience in information technology, being significantly involved in object technology adoption, requirement engineering, modeling, implementation, and IT consulting. His current interests are in organizational transition to e-business, globalization, method engineering, agile methodologies, software process reengineering, and improvement. He has a bachelor's of accounting (Cairo Uni.), G. Dip in computing (ICL, Lon), Dip in computing (Control Data, Sydney), master's of computing (UWS, Sydney), and a PhD (UTS, Sydney) in the area of migration to object oriented technology.*